SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - Database Design and Connectivity

Course Code:	DSA0501	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	**AT1 – Database Design and Connectivity**
Weightage:		**Total Marks:**	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)			
Student Name:	**S.Nandini**	**Reg. No.:**	**192473030**
Section / Batch:	**Slot A/2024**	**Date of Submission:**	
Faculty In charge:	K Veena devi	**Project Mode:**	Individual Submission

Code of Conduct

I _S.Nandini(192473030)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

CO2 – AT1 – Database Design and Connectivity

Sample Questions

1. Student Database Design and Connectivity (10 Marks)

A college wants to automate the management of student records. Design an SQLite database containing a **Student** table with appropriate fields and a primary key. Write a Python program to connect to the SQLite database, create the table if it does not exist, insert at least five student records, and display all the records stored in the table.

Aim

To design an SQLite database for storing student records and perform database operations such as creating a table, inserting records, and displaying all student details using Python.

Table Name: Student

Field Name	Data Type	Constraint
Student_ID	INTEGER	Primary Key
Name	TEXT	NOT NULL
Age	INTEGER	
Department	TEXT	
CGPA	REAL	

Python code:

```
import sqlite3
```

```
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
Student_ID INTEGER PRIMARY KEY,
Name TEXT,
Age INTEGER,
Department TEXT,
CGPA REAL)
""")
students = [
(1,"Rahul",20,"CSE",8.6),
(2,"Priya",19,"IT",9.1),
(3,"Arun",21,"ECE",8.2),
(4,"Sneha",20,"AIDS",9.0),
(5,"Kiran",22,"MECH",7.8)
]
cursor.executemany("INSERT OR IGNORE INTO Student VALUES(?,?,?,?,?)",students)
conn.commit()
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchall():
    print(row)
conn.close()
```

Output:

```
(1, 'Rahul', 20, 'CSE', 8.6)
(2, 'Priya', 19, 'IT', 9.1)
(3, 'Arun', 21, 'ECE', 8.2)
(4, 'Sneha', 20, 'AIDS', 9.0)
(5, 'Kiran', 22, 'MECH', 7.8)
```

2. Library Management Database (10 Marks)

A library plans to maintain information about its books using an SQLite database. Design a **Book** table containing suitable attributes such as Book ID, Title, Author, Publisher, and Price. Write a Python program to establish a connection with the database, create the table, insert sample records, update the price of a selected book, and retrieve the updated information.

Aim

To create a Book database using SQLite and perform insert, update, and retrieval operations.

Code:

```
import sqlite3
conn=sqlite3.connect("library.db")
cur=conn.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Book(
Book_ID INTEGER PRIMARY KEY,
Title TEXT,
Author TEXT,
Publisher TEXT,
Price REAL)
""")
books=[
(1,"Python","Guido","ABC",450),
(2,"Java","James","XYZ",550),
(3,"C++","Bjarne","Pearson",600)
]
cur.executemany("INSERT OR IGNORE INTO Book VALUES(?,?,?,?,?)",books)
cur.execute("UPDATE Book SET Price=700 WHERE Book_ID=2")
conn.commit()
cur.execute("SELECT * FROM Book WHERE Book_ID=2")
print(cur.fetchone())

conn.close()
```

Output:

```
(2, 'Java', 'James', 'XYZ', 700.0)
```

3. Employee and Department Database Design (10 Marks)

A company wants to store employee information and department details in an SQLite database. Design two related tables, **Department** and **Employee**, by identifying appropriate primary and foreign keys. Write a Python program to connect to the database, create both tables, insert sample data into each table, and display the employee name along with the corresponding department name using an SQL JOIN query.

Aim

To create Department and Employee tables with primary and foreign keys and retrieve employee details using JOIN.

Code:

[3] ✓ 0s

```
import sqlite3
conn=sqlite3.connect("company.db")
cur=conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS Department(Dept_ID INTEGER PRIMARY KEY,Dept_Name TEXT)")
cur.execute("CREATE TABLE IF NOT EXISTS Employee(Emp_ID INTEGER PRIMARY KEY,Emp_Name TEXT,Dept_ID
cur.executemany("INSERT OR IGNORE INTO Department VALUES(?,?)",
[(1,"HR"),(2,"IT")])
cur.executemany("INSERT OR IGNORE INTO Employee VALUES(?,?,?)",
[(101,"John",1),(102,"Alice",2)])
conn.commit()
cur.execute("""
SELECT Emp_Name,Dept_Name
FROM Employee
JOIN Department
ON Employee.Dept_ID=Department.Dept_ID
""")
for row in cur.fetchall():
    print(row)
conn.close()
```

Output:

```
('John', 'HR')
('Alice', 'IT')
```

4. Hospital Patient Database (10 Marks)

A hospital requires a database to manage patient information. Design an SQLite database with a **Patient** table containing suitable fields and constraints. Write a Python program to connect to the database, create the table, insert patient records, update the contact number of a specified patient, delete a discharged patient's record, and display the remaining records.

Aim

Create a Patient database and perform insert, update, delete, and display operations.

Code:

```
import sqlite3
conn = sqlite3.connect("hospital.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Patient(
    Patient_ID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    Age INTEGER,
    Disease TEXT,
    Contact TEXT
)
""")
patients = [
    (1, "Rahul", 35, "Fever", "9876543210"),
    (2, "Priya", 28, "Diabetes", "9123456789"),
    (3, "Kiran", 40, "Fracture", "9988776655"),
    (4, "Arun", 45, "Asthma", "9876501234"),
    (5, "Sneha", 30, "Migraine", "9001122334")
]
cursor.executemany("INSERT OR IGNORE INTO Patient VALUES(?,?,?,?,?)", patients)
cursor.execute("""
UPDATE Patient
SET Contact='9999999999'
WHERE Patient_ID=2
""")
cursor.execute("""
DELETE FROM Patient
WHERE Patient_ID=3
""")
conn.commit()
print("Remaining Patients:\n")
cursor.execute("SELECT * FROM Patient")
for row in cursor.fetchall():
    print(row)
conn.close()
```

Output:

```
Remaining Patients:

(1, 'Rahul', 35, 'Fever', '9876543210')
(2, 'Priya', 28, 'Diabetes', '9999999999')
(4, 'Arun', 45, 'Asthma', '9876501234')
(5, 'Sneha', 30, 'Migraine', '9001122334')
```

5. Online Course Registration System (10 Marks)

An online learning platform needs to maintain information about students and the courses they have registered for. Design two related SQLite tables, **Student** and **Course_Registration**, using appropriate primary and foreign keys. Write a Python program to connect to the database, create the tables, insert sample records, and retrieve the names of students along with the courses they have registered for using an SQL JOIN operation.

Aim

Create Student and Course_Registration tables and retrieve course details using JOIN.

```python
import sqlite3
conn = sqlite3.connect("course.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
    Student_ID INTEGER PRIMARY KEY,
    Student_Name TEXT
)
""")
cursor.execute("""
CREATE TABLE IF NOT EXISTS Course_Registration(
    Registration_ID INTEGER PRIMARY KEY,
    Student_ID INTEGER,
    Course_Name TEXT,
    FOREIGN KEY(Student_ID) REFERENCES Student(Student_ID)
)
""")
students = [
    (1, "Rahul"),
    (2, "Priya"),
    (3, "Arun")
]
registrations = [
    (101, 1, "Python Programming"),
    (102, 2, "Data Science"),
    (103, 3, "Artificial Intelligence")
]
cursor.executemany("INSERT OR IGNORE INTO Student VALUES(?,?)", students)
cursor.executemany("INSERT OR IGNORE INTO Course_Registration VALUES(?,?,?)", registrations)
conn.commit()
print("Student Course Details:\n")
cursor.execute("""
SELECT Student.Student_Name,
Course_Registration.Course_Name
FROM Student
JOIN Course_Registration
ON Student.Student_ID = Course_Registration.Student_ID
""")
for row in cursor.fetchall():
    print(row)
conn.close()
```

Output:

```
Remaining Patients:

(1, 'Rahul', 35, 'Fever', '9876543210')
(2, 'Priya', 28, 'Diabetes', '9999999999')
(4, 'Arun', 45, 'Asthma', '9876501234')
(5, 'Sneha', 30, 'Migraine', '9001122334')
```

6. Banking Account Management System (10 Marks)

A bank wants to maintain customer account information using an SQLite database. Design an appropriate database containing an **Account** table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert customer account details, update the account balance of a specified customer, and display all customer records.

Code:

```
4]
import sqlite3
conn = sqlite3.connect("bank.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Account(
    Account_No INTEGER PRIMARY KEY,
    Customer_Name TEXT,
    Account_Type TEXT,
    Balance REAL
)
""")
accounts = [
    (101, "Rahul", "Savings", 45000),
    (102, "Priya", "Current", 85000),
    (103, "Arun", "Savings", 65000),
    (104, "Sneha", "Current", 55000),
    (105, "Kiran", "Savings", 35000)
]
cursor.executemany("INSERT OR IGNORE INTO Account VALUES(?,?,?,?)", accounts)
cursor.execute("""
UPDATE Account
SET Balance = Balance + 10000
WHERE Account_No = 103
""")
conn.commit()
print("Customer Account Details:\n")
cursor.execute("SELECT * FROM Account")
for row in cursor.fetchall():
    print(row)
conn.close()
```

Output:

```
Customer Account Details:

(101, 'Rahul', 'Savings', 45000.0)
(102, 'Priya', 'Current', 85000.0)
(103, 'Arun', 'Savings', 75000.0)
(104, 'Sneha', 'Current', 55000.0)
(105, 'Kiran', 'Savings', 35000.0)
```

7. Inventory Management System (10 Marks)

A retail store needs to maintain product inventory using SQLite. Design a **Product** table containing Product ID, Product Name, Category, Quantity, and Unit Price. Write a Python program to create the database, insert sample products, identify products with quantity less than 10, update their stock quantity, and display the updated inventory.

Aim

Manage customer accounts using SQLite.

Code:

```python
import sqlite3
conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Product(
    Product_ID INTEGER PRIMARY KEY,
    Product_Name TEXT,
    Category TEXT,
    Quantity INTEGER,
    Unit_Price REAL
)
""")
products = [
    (1, "Laptop", "Electronics", 15, 65000),
    (2, "Mouse", "Accessories", 5, 500),
    (3, "Keyboard", "Accessories", 8, 1000),
    (4, "Monitor", "Electronics", 12, 12000),
    (5, "Printer", "Electronics", 6, 8000)
]
cursor.executemany("INSERT OR IGNORE INTO Product VALUES(?,?,?,?,?)", products)
print("Products with Quantity Less Than 10:\n")
cursor.execute("SELECT * FROM Product WHERE Quantity < 10")
for row in cursor.fetchall():
    print(row)
cursor.execute("""
UPDATE Product
SET Quantity = Quantity + 10
WHERE Quantity < 10
""")
conn.commit()
print("\nUpdated Inventory:\n")
cursor.execute("SELECT * FROM Product")
for row in cursor.fetchall():
    print(row)
conn.close()
```

Code:

```
Products with Quantity Less Than 10:

(2, 'Mouse', 'Accessories', 5, 500.0)
(3, 'Keyboard', 'Accessories', 8, 1000.0)
(5, 'Printer', 'Electronics', 6, 8000.0)

Updated Inventory:

(1, 'Laptop', 'Electronics', 15, 65000.0)
(2, 'Mouse', 'Accessories', 15, 500.0)
(3, 'Keyboard', 'Accessories', 18, 1000.0)
(4, 'Monitor', 'Electronics', 12, 12000.0)
(5, 'Printer', 'Electronics', 16, 8000.0)
```

8. Course and Faculty Management System (10 Marks)

A university wants to maintain details of faculty members and the courses they teach. Design two related tables, **Faculty** and **Course**, using appropriate primary and foreign keys. Write a Python program to create the database, insert sample records, and display each faculty member along with the courses assigned using an SQL JOIN query.

Aim

Create Faculty and Course tables and retrieve faculty-course details using JOIN.

```python
import sqlite3
conn = sqlite3.connect("university.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Faculty(
    Faculty_ID INTEGER PRIMARY KEY,
    Faculty_Name TEXT
)
""")
cursor.execute("""
CREATE TABLE IF NOT EXISTS Course(
    Course_ID INTEGER PRIMARY KEY,
    Course_Name TEXT,
    Faculty_ID INTEGER,
    FOREIGN KEY(Faculty_ID) REFERENCES Faculty(Faculty_ID)
)
""")
faculty = [
    (1, "Dr. Kumar"),
    (2, "Dr. Meena"),
    (3, "Dr. Ravi")
]
courses = [
    (101, "Python Programming", 1),
    (102, "Database Management", 2),
    (103, "Data Analytics", 3)
]
cursor.executemany("INSERT OR IGNORE INTO Faculty VALUES(?,?)", faculty)
cursor.executemany("INSERT OR IGNORE INTO Course VALUES(?,?,?)", courses)
conn.commit()
print("Faculty and Courses:\n")
cursor.execute("""
SELECT Faculty.Faculty_Name,
Course.Course_Name
FROM Faculty
JOIN Course
ON Faculty.Faculty_ID = Course.Faculty_ID
""")
for row in cursor.fetchall():
    print(row)
conn.close()
```

Code:

```
Faculty and Courses:

('Dr. Kumar', 'Python Programming')
('Dr. Meena', 'Database Management')
('Dr. Ravi', 'Data Analytics')
```

9. Vehicle Registration Database (10 Marks)

A transport department plans to maintain vehicle registration details in an SQLite database. Design a **Vehicle** table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert vehicle records, search for a vehicle using its registration number, and display the retrieved information.

Aim

Maintain vehicle registration information using SQLite.

Code:

```
import sqlite3
conn = sqlite3.connect("vehicle.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Vehicle(
    Registration_No TEXT PRIMARY KEY,
    Owner_Name TEXT,
    Vehicle_Type TEXT,
    Model TEXT,
    Year INTEGER
)
""")
vehicles = [
    ("TN09AB1234", "Rahul", "Car", "Hyundai i20", 2023),
    ("TN10CD5678", "Priya", "Bike", "Honda Activa", 2022),
    ("TN11EF4321", "Arun", "Car", "Maruti Swift", 2024)
]
cursor.executemany("INSERT OR IGNORE INTO Vehicle VALUES(?,?,?,?,?)", vehicles)
conn.commit()
reg_no = "TN09AB1234"
cursor.execute("""
SELECT * FROM Vehicle
WHERE Registration_No = ?
""", (reg_no,))
record = cursor.fetchone()
print("Vehicle Details:\n")
if record:
    print(record)
else:
    print("Vehicle not found")
conn.close()
```

Output:

```
••• Vehicle Details:

    ('TN09AB1234', 'Rahul', 'Car', 'Hyundai i20', 2023)
```

10. Hospital Appointment Management System (10 Marks)

A hospital maintains separate tables for **Doctors** and **Appointments**. Design both tables using appropriate primary and foreign keys. Write a Python program to create the tables, insert sample records, and display the doctor name along with the appointment details using a JOIN query.

Aim

Create Doctor and Appointment tables and retrieve appointment details using JOIN.

```
import sqlite3
conn = sqlite3.connect("appointment.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Doctor(
    Doctor_ID INTEGER PRIMARY KEY,
    Doctor_Name TEXT,
    Specialization TEXT
)
""")
cursor.execute("""
CREATE TABLE IF NOT EXISTS Appointment(
    Appointment_ID INTEGER PRIMARY KEY,
    Patient_Name TEXT,
    Doctor_ID INTEGER,
    Appointment_Date TEXT,
    FOREIGN KEY(Doctor_ID) REFERENCES Doctor(Doctor_ID)
)
""")
doctors = [
    (1, "Dr. Rajesh", "Cardiology"),
    (2, "Dr. Priya", "Dermatology"),
    (3, "Dr. Kumar", "Orthopedics")
]
appointments = [
    (101, "Rahul", 1, "15-07-2026"),
    (102, "Sneha", 2, "16-07-2026"),
    (103, "Arun", 3, "17-07-2026")
]
cursor.executemany("INSERT OR IGNORE INTO Doctor VALUES(?,?,?)", doctors)
cursor.executemany("INSERT OR IGNORE INTO Appointment VALUES(?,?,?,?)", appointments)
conn.commit()
print("Appointment Details:\n")
cursor.execute("""
SELECT Doctor.Doctor_Name,
Appointment.Patient_Name,
Appointment.Appointment_Date
FROM Doctor
JOIN Appointment
ON Doctor.Doctor_ID = Appointment.Doctor_ID
""")
for row in cursor.fetchall():
    print(row)
conn.close()
```

Output:

```
Vehicle Details:

('TN09AB1234', 'Rahul', 'Car', 'Hyundai i20', 2023)
```